Лабораторная работа №6 "Интеграция с NoSQL базой данных MongoDB"

Лабораторная работа №6. Интеграция с NoSQL базой данных MongoDB

Ветка репозитория: `26`

---

1. ЦЕЛЬ РАБОТЫ

Освоение работы с NoSQL базой данных MongoDB: подключение, создание схем и моделей с помощью ODM Mongoose, выполнение CRUD-операций, использование документной модели данных (вложенные документы, массивы) **для альтернативной реализации хранения данных вашего курсового проекта** (в дополнение к реляционной PostgreSQL).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы документно-ориентированных баз данных.
- MongoDB (коллекции, документы, BSON, ObjectId).
- ODM Mongoose (схема, модель, методы).
- Строку подключения, работу с MongoDB Atlas.
- Отличия MongoDB от реляционных СУБД (PostgreSQL).

---

3. ЗАДАЧИ РАБОТЫ

- Создать бесплатный кластер в MongoDB Atlas (или использовать локальную установку).
- Подключить Mongoose к Node.js-приложению (существующему из предыдущих работ или новому).

- Определить схему и модель для коллекции, **соответствующей предметной области вашего курсового проекта**.
- Реализовать CRUD-операции через Express-маршруты.
- Использовать вложенные документы или массивы (например, массив тегов, список комментариев, корзину товаров) – в соответствии с логикой вашего проекта.
- Продемонстрировать отличие от реляционной модели (например, показать, как те же данные можно хранить в виде документа с вложенными структурами, а не в нескольких таблицах).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Настройка MongoDB

1. Зарегистрируйтесь на MongoDB Atlas (бесплатный тариф).
2. Создайте бесплатный кластер (M0).
3. Создайте пользователя базы данных, запомните пароль.
4. Получите строку подключения (Connection String) – она понадобится в приложении.

   Альтернатива: установите MongoDB Community Edition локально – все инструкции и код будут теми же (изменится только строка подключения).

Часть 2. Установка зависимостей

```
npm install mongoose
```

(Рекомендуется также установить `dotenv` для хранения строки подключения в `.env`.)

Часть 3. Подключение к базе данных

В файле `server.js` (или `db.js`) добавьте:

```
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error('MongoDB connection error:', err));
```

Часть 4. Создание схемы и модели

Создайте папку `models`, файл `<ModelName>.js`. Определите схему **в соответствии с вашей предметной областью**. Обязательно используйте хотя бы один вложенный документ или массив.

Пример для интернет-магазина (товар с отзывами):

```
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: Number,
  category: String,
  reviews: [{ userId: String, rating: Number, comment: String }],
  tags: [String],
  createdAt: { type: Date, default: Date.now }
});
const Product = mongoose.model('Product', productSchema);
module.exports = Product;
```

Часть 5. Реализация CRUD-операций

Создайте маршруты (Express), в которых используются методы Mongoose:

- `GET /items` → `Model.find()`
- `GET /items/:id` → `Model.findById(id)`
- `POST /items` → `new Model(body).save()` или `Model.create(body)`
- `PUT /items/:id` → `Model.findByIdAndUpdate(id, body, { new: true })`
- `DELETE /items/:id` → `Model.findByIdAndDelete(id)`

Часть 6. Работа с вложенными структурами

- Добавьте хотя бы один эндпоинт, который использует вложенные массивы (например, добавление отзыва к товару, добавление подзадачи, регистрация участника на событие).
- Продемонстрируйте, как MongoDB позволяет обновлять элемент внутри массива (например, увеличить количество товара в корзине).

Часть 7. Проверка и тестирование

- Запустите сервер.
- Используйте Postman или аналогичный инструмент для тестирования CRUD-эндпоинтов.
- Проверьте данные в MongoDB Compass или веб-интерфейсе Atlas (покажите вложенную структуру).

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист, цель работы.
- Сравнение реляционных (SQL) и документных (NoSQL) баз данных (краткие теоретические сведения).
- **Скриншоты:**
    - дашборд MongoDB Atlas (созданный кластер);
    - строка подключения (URI) в файле `.env` или `config`;
    - схема и модель (листинг кода);
    - примеры запросов в Postman (GET, POST, PUT, DELETE);
    - данные в MongoDB Compass или Atlas Data Explorer (показать вложенную структуру);
- Листинг ключевых частей кода (подключение, модель, маршруты).
- Ответы на контрольные вопросы (16 вопросов).
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка **26**).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов.**
- Код должен быть загружен в репозиторий с веткой `26`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. Чем MongoDB отличается от реляционных СУБД (PostgreSQL, MySQL)? Назовите минимум 4 отличия.
2. Что такое `ObjectId`? Как он генерируется и для чего используется?
3. Как выполнить поиск документов по вложенному полю (например, найти пользователей, чей `profile.city` равен "Минск")?
4. Какие типы данных поддерживает BSON (бинарный JSON)? Назовите не менее 5 типов.
5. Что такое агрегация (aggregation pipeline) в MongoDB? Приведите пример (хотя бы двух стадий).
6. Как в Mongoose определить вложенный документ (subdocument) без отдельной коллекции? Приведите пример из вашего проекта.
7. Как выполнить обновление элемента внутри массива (например, увеличить количество товара в корзине)?
8. В чём преимущество использования вложенных документов перед отдельными коллекциями? В чём недостаток?
9. Как в Mongoose создать индекс для поля (например, для ускорения поиска по `email`)?
10. Что такое `populate` в Mongoose и для чего он нужен? (если ссылаетесь на другую коллекцию)

11. Как в MongoDB выполнить поиск с условием «поле массива содержит значение» (например, `tags` содержит "важный")?
12. Как удалить документ по `id` и вернуть удалённый документ в ответе?
13. Какие методы Mongoose возвращают `Query` и позволяют строить цепочки (например, `.where().limit().sort()`)?
14. Как в Mongoose добавить кастомный метод экземпляра (instance method) или статический метод модели?
15. Как в MongoDB и Mongoose обеспечить каскадное удаление (при удалении родителя удалить все связанные дочерние документы)?
16. Для каких типов приложений MongoDB подходит лучше, чем реляционные базы, а для каких – хуже? Обоснуйте применительно к вашему курсовому проекту.

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Данная лабораторная работа может выполняться **как отдельный проект** (чтобы не смешивать с PostgreSQL), либо как параллельная реализация того же API с использованием MongoDB (в отдельной ветке). Рекомендуется выделить отдельную ветку `26` и создать копию сервера, заменив Sequelize на Mongoose.
- В курсовом проекте вы сможете выбрать, какую базу данных использовать (реляционную или документную). Эта работа даёт вам альтернативный опыт.
- **Доступность в Беларуси:** MongoDB Atlas бесплатен и должен быть доступен (возможны временные затруднения, но обычно работает). При недоступности используйте локальную установку MongoDB Community Edition.
- Для тестирования рекомендовано использовать Postman, а для просмотра данных – MongoDB Compass (его можно установить локально или использовать встроенный просмотрщик Atlas).

---

9. КРИТЕРИИ ОЦЕНКИ

- Настройка MongoDB Atlas и успешное подключение к приложению.

- Корректное определение схемы и модели (соответствие предметной области курсового проекта, использование вложенных структур).
- Реализация всех CRUD-операций (GET список, GET по id, POST, PUT/PATCH, DELETE) с использованием Mongoose.
- Использование вложенных документов или массивов (соответствует логике проекта).
- Качество кода: структура проекта, использование переменных окружения, обработка ошибок.
- Полнота отчёта: скриншоты, листинги, ответы на контрольные вопросы.
- Ссылка на репозиторий с веткой `26` и инструкцией по запуску.